FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Don Marco M. Sison
Section:	AN#42
Professor:	Ms. Crisola G. Tan
Date:	30/09/2026

Github repo link: https://github.com/dmsison-ph/Sison-DonMarco-CS0065-AN42

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# -------- Run Model -----------
```

```
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation ----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

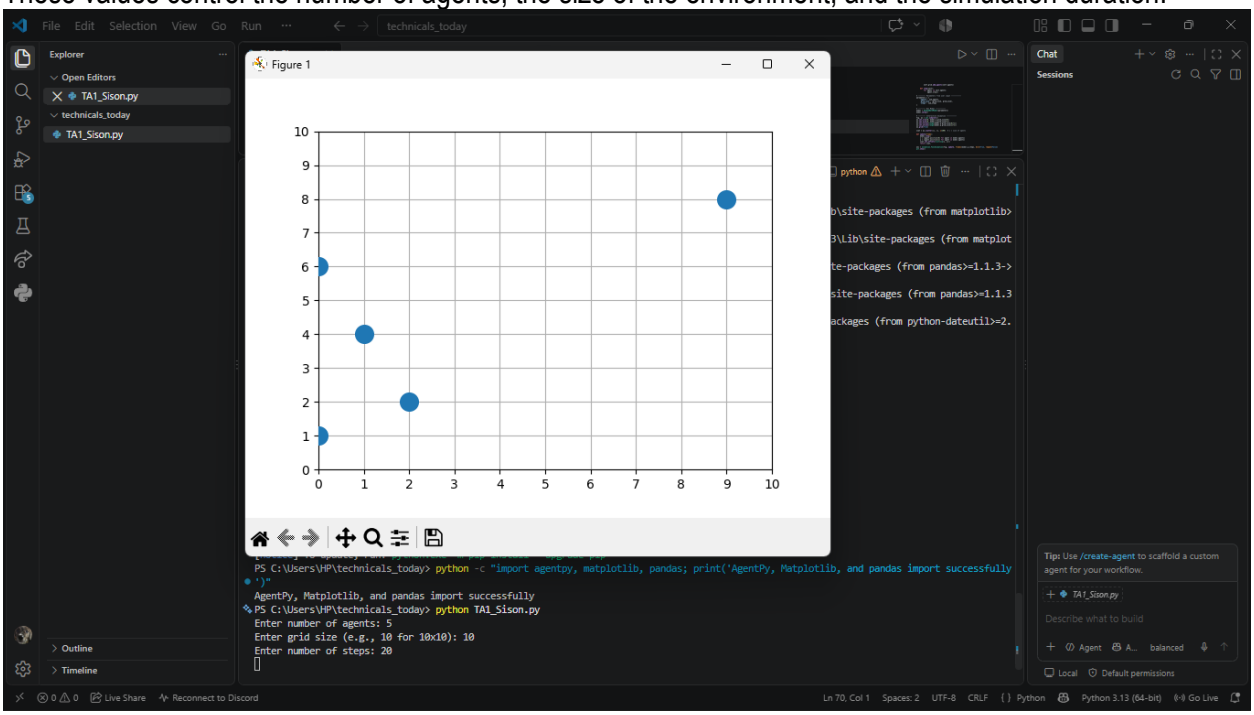
scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

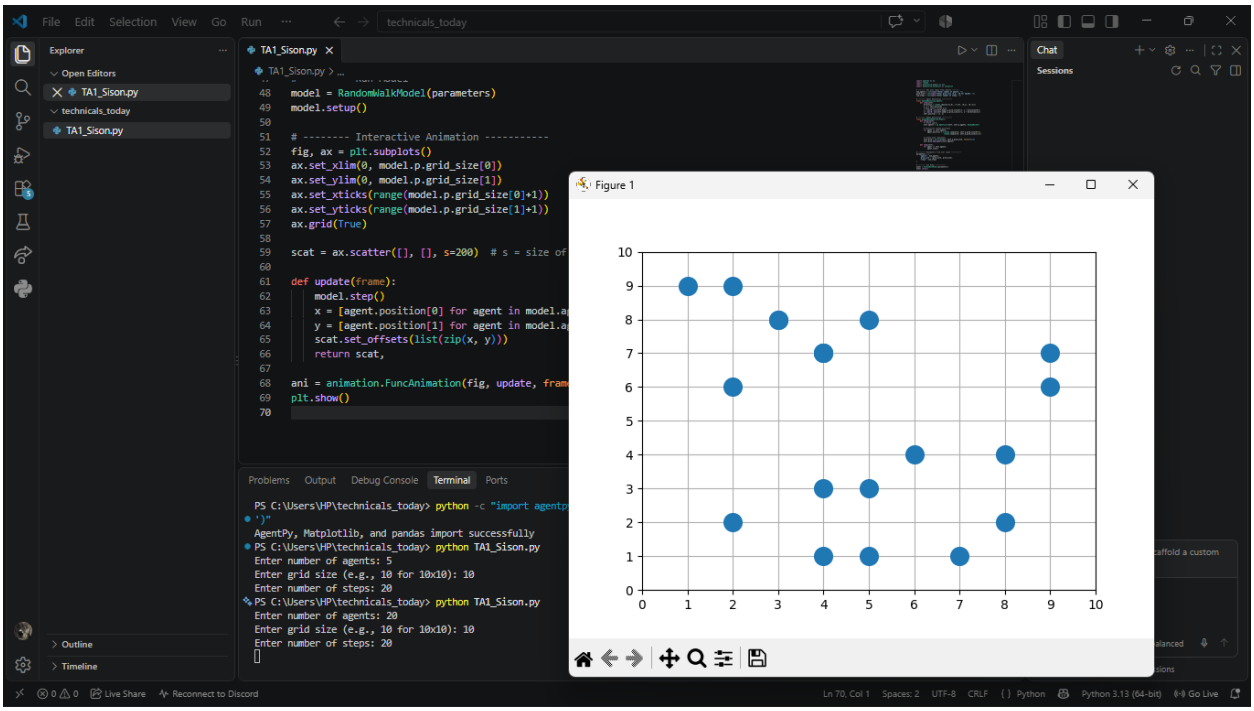


Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

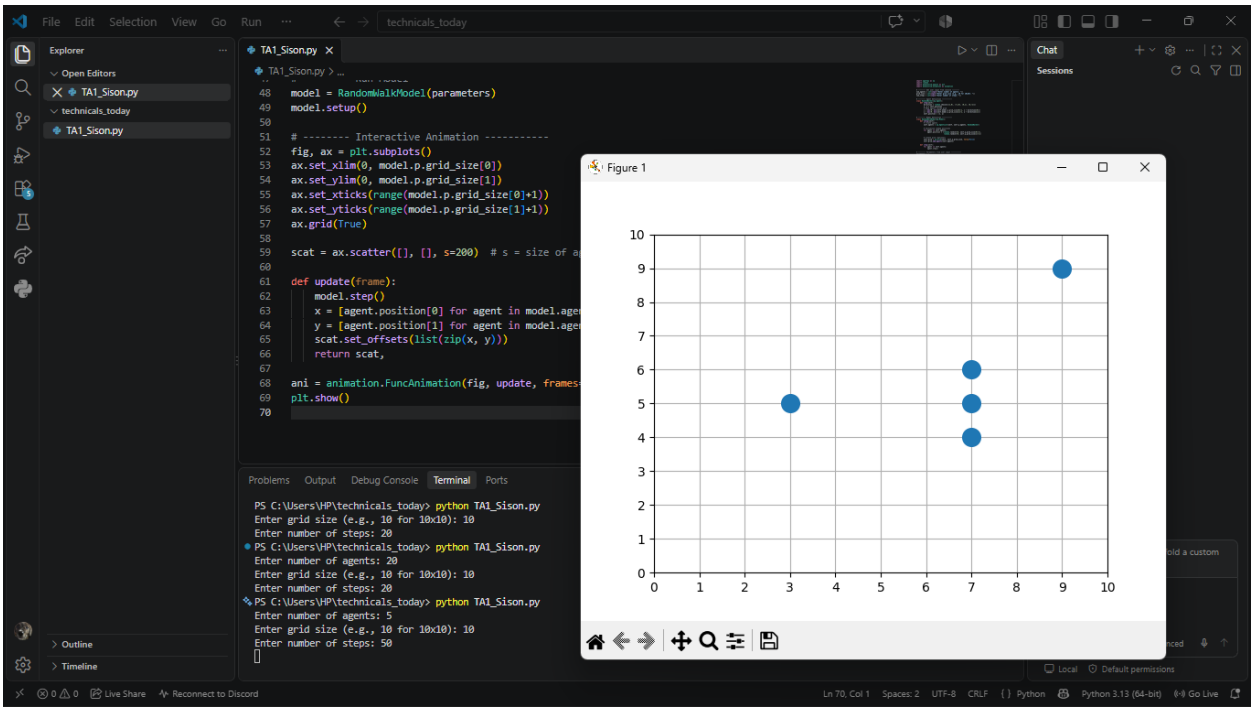
Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely



Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.



Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks)⇩

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
```

```
        direction = random.choices([(1, 0), (-1, 0), (0, 1), (0, -1)], weights=[3, 1, 1, 1], k=1)[0]
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)
        self.path.append(self.position)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))
            agent.path = [agent.position]

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation -----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
```

```
ax.grid(True)

scat = ax.scatter([], [], s=200)

path_lines = [ax.plot([], [], linewidth=1.5, label=f"Agent {i}")[0]
              for i in range(1, num_agents + 1)]

def init():
    positions = [agent.position for agent in model.agents]
    scat.set_offsets(positions)
    return scat,

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    for agent, line in zip(model.agents, path_lines):
        path_x = [position[0] for position in agent.path]
        path_y = [position[1] for position in agent.path]
        line.set_data(path_x, path_y)
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, init_func=init, blit=False, repeat=False)
ax.legend()
plt.show()

for number, agent in enumerate(model.agents, start=1):
    print(f"Agent {number} path: {agent.path}")

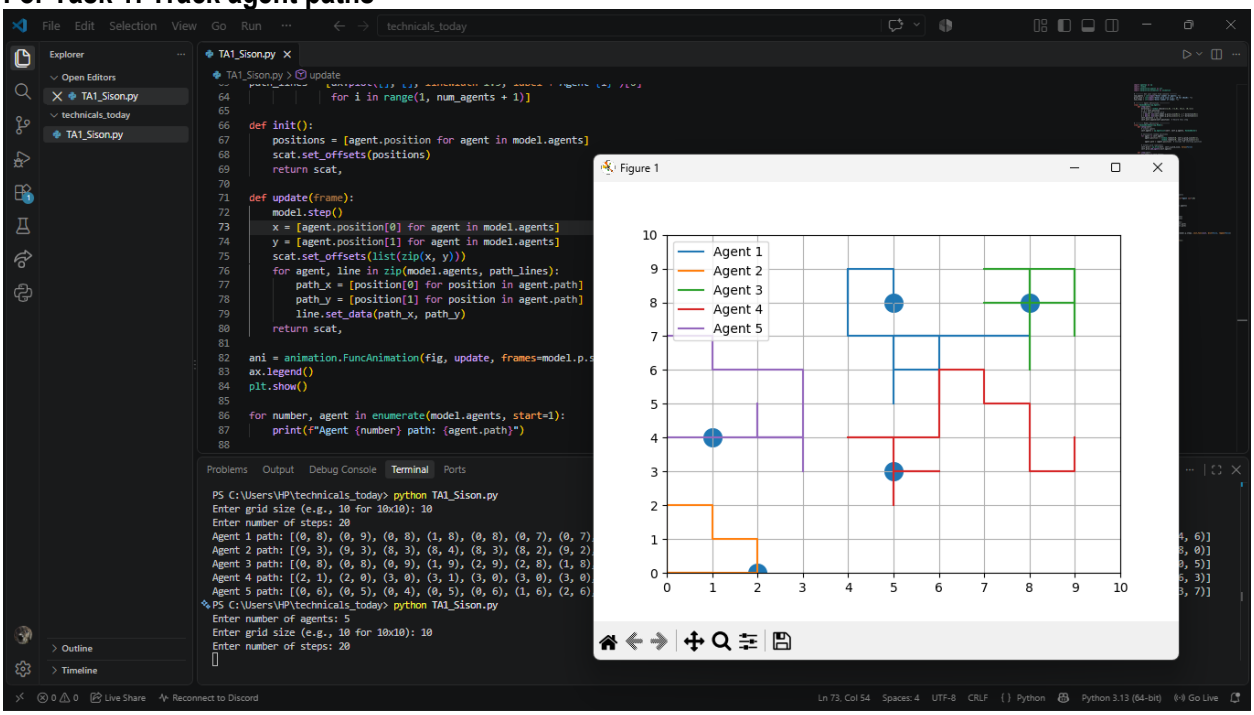
from collections import Counter

final_positions = [agent.position for agent in model.agents]
position_counts = Counter(final_positions)

print("\nFinal position distribution:")
for position, count in sorted(position_counts.items()):
    print(f"{position}: {count} agent(s)")

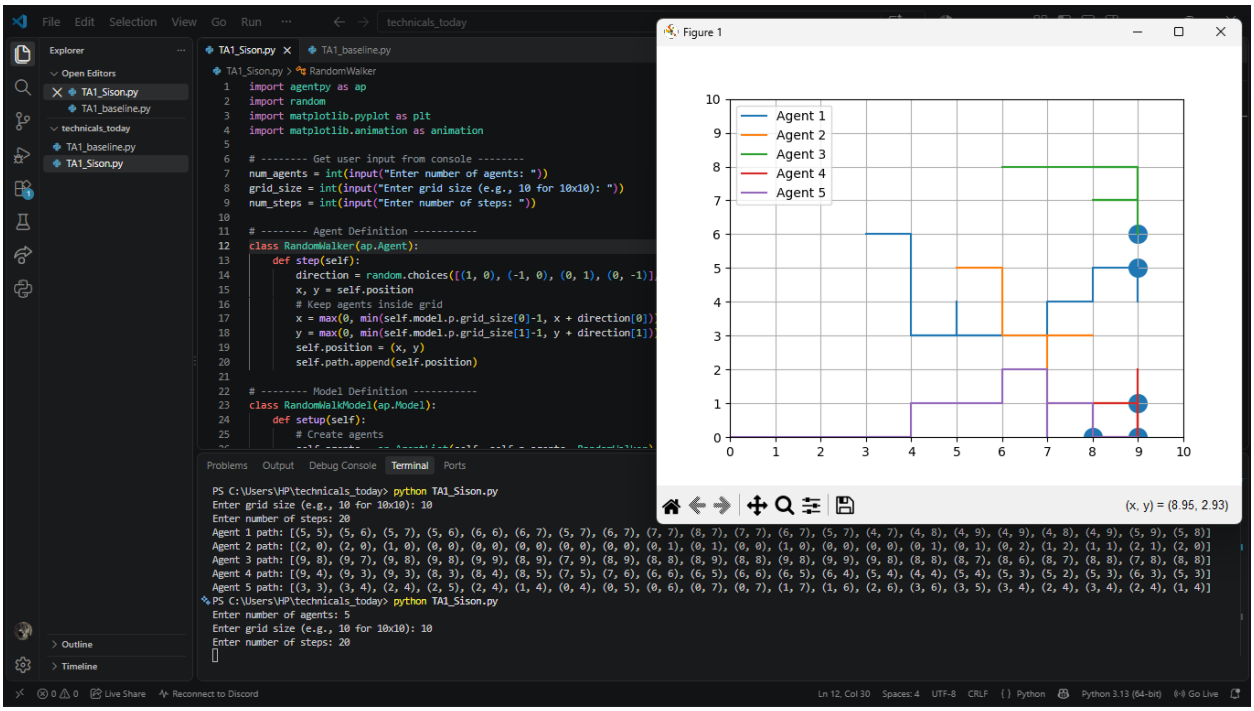
average_x = sum(x for x, y in final_positions) / len(final_positions)
print(f"Average final x-position: {average_x:.2f}")
```

For Task 1: Track agent paths




```
PS C:\Users\HP\technicals_today> python TA1_Sison.py
Enter number of agents: 5
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 20
Agent 1 path: [(5, 5), (5, 6), (5, 7), (5, 6), (6, 6), (6, 7), (5, 7), (6, 7), (7, 7), (8, 7), (7, 7), (6, 7), (5, 7), (4, 7), (4, 8), (4, 9), (4, 9), (4, 8), (4, 9), (5, 9), (5, 8)]
Agent 2 path: [(2, 0), (2, 0), (1, 0), (0, 0), (0, 0), (0, 0), (0, 0), (0, 0), (0, 1), (0, 1), (0, 0), (1, 0), (0, 0), (0, 0), (0, 1), (0, 1), (0, 2), (1, 2), (1, 1), (2, 1), (2, 0)]
Agent 3 path: [(9, 8), (9, 7), (9, 8), (9, 8), (9, 9), (8, 9), (7, 9), (8, 9), (8, 8), (8, 9), (8, 8), (9, 8), (9, 9), (9, 8), (8, 8), (8, 7), (8, 6), (8, 7), (8, 8), (7, 8), (8, 8)]
Agent 4 path: [(9, 4), (9, 3), (9, 3), (8, 3), (8, 4), (8, 5), (7, 5), (7, 6), (6, 6), (6, 5), (6, 6), (6, 5), (6, 4), (5, 4), (4, 4), (5, 4), (5, 3), (5, 2), (5, 3), (6, 3), (5, 3)]
Agent 5 path: [(3, 3), (3, 4), (2, 4), (2, 5), (2, 4), (1, 4), (0, 4), (0, 5), (0, 6), (0, 7), (0, 7), (1, 7), (1, 6), (2, 6), (3, 6), (3, 5), (3, 4), (2, 4), (3, 4), (2, 4), (1, 4)]
PS C:\Users\HP\technicals_today>
```

For Task 2:

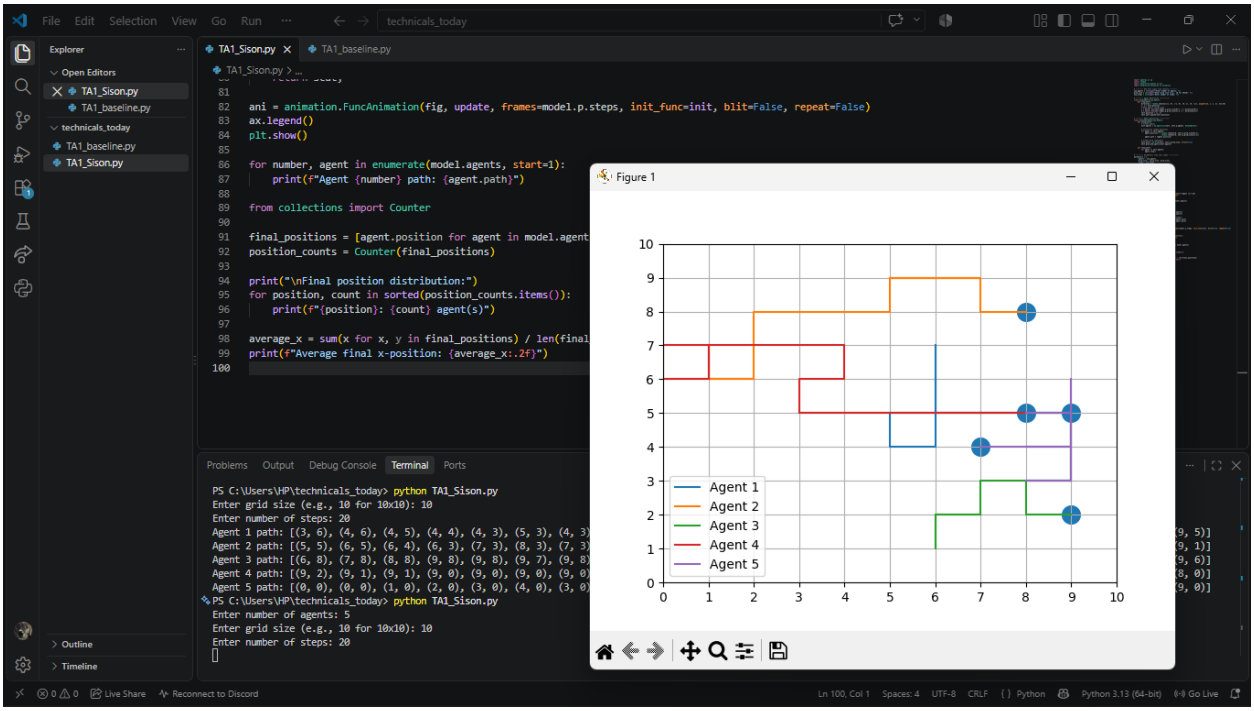


python TA1_Sison.py
Enter number of agents: 5
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 20
Agent 1 path: [(3, 6), (4, 6), (4, 5), (4, 4), (4, 3), (5, 3), (4, 3), (5, 3), (5, 4), (5, 3), (6, 3), (5, 3), (6, 3), (7, 3), (7, 4), (8, 4), (8, 5), (9, 5), (9, 4), (9, 5), (9, 5)]
Agent 2 path: [(5, 5), (6, 5), (6, 4), (6, 3), (7, 3), (8, 3), (7, 3), (7, 2), (7, 1), (8, 1), (9, 1), (9, 2), (9, 2), (9, 2), (9, 1), (9, 1), (9, 1), (9, 0), (9, 1), (8, 1), (9, 1)]
Agent 3 path: [(6, 8), (7, 8), (8, 8), (9, 8), (9, 8), (9, 7), (9, 8), (9, 8), (8, 8), (9, 8), (9, 7), (8, 7), (9, 7), (9, 7), (8, 7), (9, 7), (9, 7), (9, 7), (9, 7), (9, 7), (9, 6)]
Agent 4 path: [(9, 2), (9, 1), (9, 1), (9, 0), (9, 0), (9, 0), (9, 0), (9, 0), (9, 0), (9, 0), (9, 1), (9, 0), (9, 0), (8, 0), (9, 0), (9, 0), (9, 1), (9, 1), (9, 1), (8, 1), (8, 0)]
Agent 5 path: [(0, 0), (0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (3, 0), (4, 0), (4, 0), (4, 1), (5, 1), (6, 1), (6, 2), (7, 2), (7, 1), (7, 0), (7, 0), (7, 1), (8, 1), (8, 0), (9, 0)]

For Task 3: Final Positions

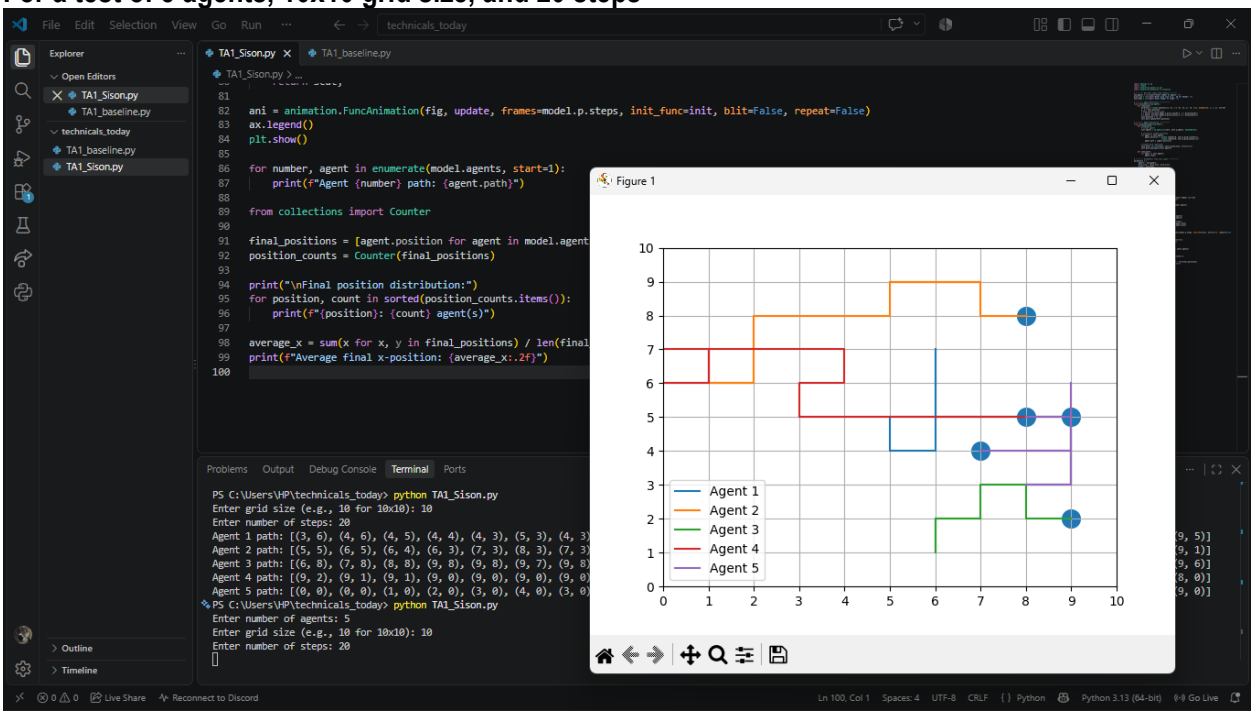
```
PS C:\Users\HP\technicals_today> python TA1_Sison.py
Enter number of agents: 5
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 20
Agent 1 path: [(3, 5), (4, 5), (5, 5), (6, 5), (6, 4), (5, 4), (5, 5), (6, 5), (7, 5), (6, 5), (6, 6), (6, 7), (6, 6), (6, 5), (7, 5), (8, 5), (9, 5), (9, 5), (8, 5), (7, 5), (8, 5)]
Agent 2 path: [(0, 7), (0, 6), (0, 6), (1, 6), (2, 6), (1, 6), (2, 6), (2, 7), (2, 8), (3, 8), (4, 8), (5, 8), (5, 9), (6, 9), (5, 9), (6, 9), (7, 9), (6, 9), (7, 9), (7, 8), (8, 8)]
Agent 3 path: [(7, 2), (6, 2), (6, 1), (6, 2), (7, 2), (7, 3), (8, 3), (9, 3), (9, 4), (9, 4), (9, 4), (9, 4), (9, 5), (9, 5), (9, 4), (9, 3), (8, 3), (8, 2), (9, 2), (9, 2), (9, 2)]
Agent 4 path: [(1, 7), (1, 6), (0, 6), (0, 7), (1, 7), (1, 6), (1, 7), (2, 7), (3, 7), (4, 7), (4, 6), (3, 6), (3, 5), (4, 5), (5, 5), (6, 5), (7, 5), (8, 5), (9, 5), (9, 5), (9, 5)]
Agent 5 path: [(8, 5), (9, 5), (9, 5), (9, 6), (9, 5), (9, 5), (9, 5), (9, 4), (8, 4), (9, 4), (9, 3), (9, 3), (8, 3), (9, 3), (9, 3), (9, 4), (9, 5), (9, 4), (8, 4), (7, 4)]

Final position distribution:
(7, 4): 1 agent(s)
(8, 5): 1 agent(s)
(8, 8): 1 agent(s)
(9, 2): 1 agent(s)
(9, 5): 1 agent(s)
Average final x-position: 8.20
```



For Task 4: Running and comparing two simulations

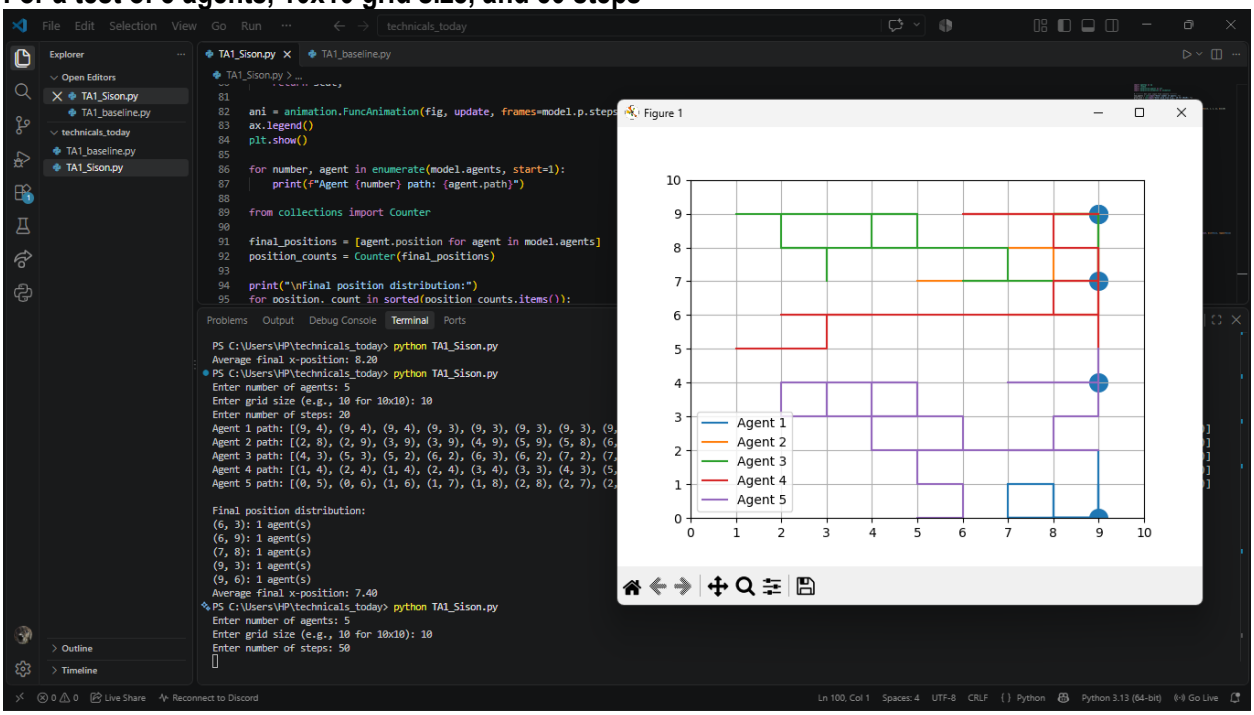
For a test of 5 agents, 10x10 grid size, and 20 steps

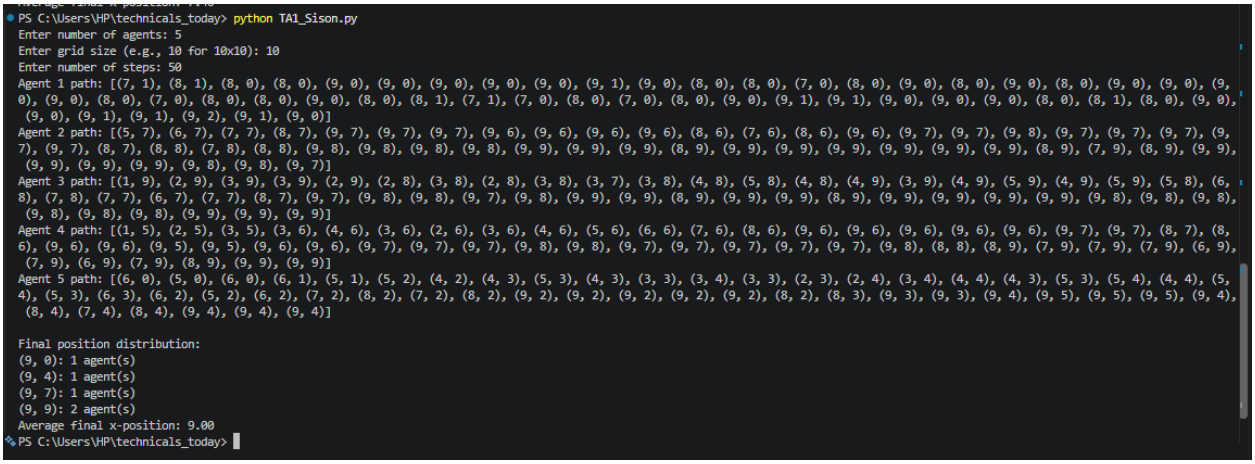



```
PS C:\Users\HP\technicals_today> python TA1_Sison.py
Enter number of agents: 5
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 20
Agent 1 path: [(3, 5), (4, 5), (5, 5), (6, 5), (6, 4), (5, 4), (5, 5), (6, 5), (7, 5), (6, 5), (6, 6), (6, 7), (6, 6), (6, 5), (7, 5), (8, 5), (9, 5), (9, 5), (8, 5), (7, 5), (8, 5)]
Agent 2 path: [(0, 7), (0, 6), (0, 6), (1, 6), (2, 6), (1, 6), (2, 6), (2, 7), (2, 8), (3, 8), (4, 8), (5, 8), (5, 9), (6, 9), (5, 9), (6, 9), (7, 9), (6, 9), (7, 9), (7, 8), (8, 8)]
Agent 3 path: [(7, 2), (6, 2), (6, 1), (6, 2), (7, 2), (7, 3), (8, 3), (9, 3), (9, 4), (9, 4), (9, 4), (9, 4), (9, 5), (9, 5), (9, 4), (9, 3), (8, 3), (8, 2), (9, 2), (9, 2), (9, 2)]
Agent 4 path: [(1, 7), (1, 6), (0, 6), (0, 7), (1, 7), (1, 6), (1, 7), (2, 7), (3, 7), (4, 7), (4, 6), (3, 6), (3, 5), (4, 5), (5, 5), (6, 5), (7, 5), (8, 5), (9, 5), (9, 5), (9, 5)]
Agent 5 path: [(8, 5), (9, 5), (9, 5), (9, 5), (9, 6), (9, 5), (9, 5), (9, 5), (9, 4), (8, 4), (9, 4), (9, 3), (9, 3), (8, 3), (9, 3), (9, 3), (9, 4), (9, 5), (9, 4), (8, 4), (7, 4)]

Final position distribution:
(7, 4): 1 agent(s)
(8, 5): 1 agent(s)
(8, 8): 1 agent(s)
(9, 2): 1 agent(s)
(9, 5): 1 agent(s)
Average final x-position: 8.20
```

For a test of 5 agents, 10x10 grid size, and 50 steps





From the two runs with one as the 20-step and the other as the 50-step simulation, the 50-step has produced longer paths and all agents ended at the right boundary because the movement rule favored rightward steps in both scenarios. These runs began at random positions and uses a series of different random movies that describes the observed runs rather than proving that every 50-step run will have a higher final average.

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

Answer the Questions ⇩

1. What happens if the number of agents is very high compared to the grid size?

- When the number of agents is high compared with the grid size, multiple agents may occupy the same position and appear as one dot. One example I had was 50 agents on a 5 × 5 grid that would cause some overlapping positions because the grid has only 25 positions.

2. How does increasing the number of steps affect the spread of agents?

- If you increase the steps to spread agents, it shows more chances and will cover more positions, though the final positions aren't guaranteed to be further apart since agents can revisit positions or may overlap to the same position of an agent, where my 50-step run is where all five of my agents finished at the right boundary because the movement was likely moving to the right.

3. Do the agents ever move outside the grid? Why or why not?

- No, because the grid only allows you the agents to move at its cap size like 5x5 grid so therefore it will only be limited to the grid size you've input.

4. How can this simulation be used to model real-life phenomena (e.g., diffusion, crowd movement)?

- In real life, ants can be an example where they wander around, the ant represents the agent, and the path is where they travel looking for food. Multiple ants are considered a crowd of ants finding through different areas until they find food.